

Локализация JavaScript/TypeScript

Введение

Подключение функции rk в контроле

В первую очередь нужно подключить функцию для перевода с помощью RequireJS плагин i18n!. Зависимость должна содержать имя интерфейсного модуля, чей словарь вы хотите использовать.

```
1 import * as rk from 'i18n!Controls-Buttons';
```

Локализация на сервере

При построении контролов на сервере следует иметь в виду, что прототип контрола будет общим для всех языков, на которые будет осуществляться перевод. Поэтому существует ограничение на объявление локализуемых свойств класса: нельзя использовать конструкции, выполняющие приведение типов, например, конкатенацию строк:

```
1 caption: rk('Скачать') + '...'
```

Нужно, чтобы переводимая фраза целиком передавалась аргументом в функцию rk():

```
1 caption: rk('Скачать...')
```

Связано это с тем, что на сервере функция rk() возвращает не обычную строку, а объект, строковое значение которого вычисляется "лениво" при приведении к типу "Строка". Поэтому прототип класса должен хранить переводимые свойства в экземплярах подобных объектов, а не строк.

Подробнее о подключении словарей переводов в рагах читайте в [статье](#).

Локализация текста с версткой

Когда стоит задача выполнить перевод строки, часть которой является версткой (слово/фраза с подчеркиванием, гиперссылка и пр.), возникает необходимость отделить эту часть от обычного текста. Ранее разработчики самостоятельно разбивали текст на ключи и переводили их по отдельности. Часто это приводило к тому, что при переводе на другой язык страдал смысл текста или порядок слов в предложении нарушался (порядок слов в одном и том же по смыслу предложении для разных языков может отличаться).

Чтобы в результате перевода контекст сохранялся, платформой было реализовано следующее решение: для строк с версткой появляется специализированный ключ с пометкой **template#**. В качестве перевода устанавливают строку с обозначенным интервалом, текст из которого нужно будет преобразовать в верстку (например, выделить подчеркиванием или сделать гиперссылкой). Эта шаблонная строка будет передана в специальную функцию **splitTemplateString**, которая разобьет строку на подстроки для дальнейшего перевода.

Рассмотрим пример.

Словарь ru.json:

```
1 {
2     // Словосочетание "рабочее место" в данном примере является
   гиперссылкой.
3     "template#Укажите рабочее место": "Укажите рабочее место::{8-21}"
4 }
```

Словарь kz.json:

```

1 {
2     // Словосочетание "рабочее место" при переводе на казахский
    перемещается в начало предложения.
3     "template#Укажите рабочее место": "Жұмыс орнын корсенетизе:::{0-
11}"
4 }

```

Функция **splitTemplateString**:

```

1 import { splitTemplateString } from "Types/util";
2 import * as rk from "i18n!ModuleName";
3
4 const templateStr = rk('Укажите рабочее место', undefined, undefined,
    true);
5 // для ru - ['Укажите ', 'рабочее место', ''];
6 // для kz - ['', 'Жұмыс орнын', 'корсенетизе'];
7 const [start, link, end] = splitTemplateString(templateStr);

```

В результате функция вернет массив, количество элементов в котором рассчитывается по формуле $2X+1$, где X - это количество выделенных интервалов в переводимой строке. Интервал не может быть первым или последним элементом в массиве.

Рассмотрим строку с несколькими гиперссылками:

```

1 {
2     // Словосочетание "рабочее место" и "ФИО" в данном примере являются
    гиперссылками.
3     "template#Укажите рабочее место и ФИО сотрудника": "Укажите рабочее
    место и ФИО сотрудника:::{8-21}{24-27}"
4 }

```

Результатом выполнения функции **splitTemplateString** будет массив - ["Укажите ", "рабочее место", " и ", "ФИО", " сотрудника"].

Конкатенация строк

В исходном JavaScript коде можно встретить склеенные строки следующим образом:

```

1 throw Error(rk('Ошибка: ') + message + rk(' с кодом ') + code);

```

Из-за того, что строки формируются динамически, и в `rk` нельзя передавать переменные, для их локализации рекомендуется использовать функцию [template](#):

```

1 import {template} from 'Types/formatter';
2
3 const errorString = rk('Задача №${id} уже находится в статусе
    "${status}"');
4 throw new Error(template(errorString, {
5     id: 42,
6     status: rk('Выполнение')
7 }));

```

При локализации, например, на английский язык, в словаре переводов **en.json** ключи локализации и его перевод будут такими:

```

1 {
2     "Задача №${id} уже находится в статусе \"${status}\"" : "The task
    with id ${id} has \"${status}\" state already",

```

```
3     "Выполнение": "In progress"
4 }
```

Локализация дат

Используйте [Types/formatter:date](#).

```
1 import {date as format} from 'Types/formatter';
2
3 export default function Component({ article }) {
4     return <div>{format(article.created, format.FULL_DATETIME)}</div>;
5 }
```

Склонение слова в зависимости от числа

В функции `rk()` доступен необязательный числовой параметр, от значения которого можно установить соответствующую форму слова для текущей локализации.

```
1 // новый формат вызова функции
2 rk(stringToLocalization, contextString, number)
3
4 // второй вариант вызова функции, где второй аргумент number – число
5 rk(stringToLocalization, number)
```

При склонении слова в зависимости от числа предполагается, что исходная строка `stringToLocalization` будет следующего вида: запись(-и, -ей) . К исходной строке добавляются скобки с вариантами окончания слова. Такой вид строки нужен только для того, чтобы выделить её среди остальных в [словаре переводов](#). Добавленные в скобках окончания никак не повлияют на перевод или форму слова при локализации. Если на сервисе отключена локализация, то строка будет выводиться в исходном виде.

Чтобы через функцию `rk()` добавить склонение слова в зависимости от числа, Вам нужно:

1. В исходном коде обернуть строку функцией `rk()`.

```
1 // число, которое соответствует форме склоняемого слова
2 import { template } from 'Types/formatter';
3 import * as rk from 'i18n!ModuleName';
4
5 const numberOfRecords = 3;
6 const result = template(
7     rk('В таблице БД хранится ${num} ${row}'),
8     {
9         num: numberOfRecords
10        row: rk("запись(-и, -ей)", numberOfRecords)
11    }
12 );
```

2. В словарях перевода контрола: **ru.json** (для локализации на русском языке), **en.json** (для локализации на английском языке) и других — добавить "ключ:значение" (см. [Правила](#)):

```
1 /* ru.json */
2 {
3     "plural#запись(-и, -ей)": "запись | записи | записей | записи"
4 }
5
6 /* en.json */
7 {
8     "plural#запись(-и, -ей)": "record | records"
9 }
10
11 /* en.json */
```

```

12 | {
13 |     "В таблице БД хранится ${num} ${row}": "The database table stores
    |     ${num} ${row}"
14 | }

```

В этом случае вызов функции `rk('запись(-и,-ей)', 21)` вернет для локализации:

- на русском языке — "запись";
- на английском языке — "records";
- на стенде без локализации — "запись(-и,-ей)".

Правила добавления строки в словарь переводов для назначения разных форм слова и их локализации

В словарь переводов локализуемую строку добавляют с префиксом **plural**, которые разделяются символом **#** (решётка). В качестве перевода устанавливают строку, которая содержит слова в нужных склонениях, разделённые через **|** (pipe, вертикальная линия).

```

1 | /* ru.json */
2 | "plural#запись(-и,-ей)": "запись|записи|записей|записи"
3 |
4 | /* en.json */
5 | "plural#запись(-и,-ей)": "record|records"

```

Для локализации на русском языке используются 4 слова, располагающиеся в следующем порядке:

1. для единицы. Например одна "запись" или один "день".
2. для диапазона от 2 до 4. Например две "записи" или два "дня".
3. для диапазона от 5 до 9. Например пять "записей" или пять "дней".
4. для дроби. Например полторы "записи" или полтора "дня".

Для локализации на английском языке используются два слова, расположенные в следующем порядке:

- Первое слово — для единицы. Например, one "record".
- Второе слово — для нескольких. Например, two "records".

Правила для остальных языков Вы можете найти [здесь](#). Сейчас для локализации поддерживаются только русский и английский языки.

Если в функции `rk` мы указываем контекст и количество для склонения, то в словаре должно быть записано:

```

1 | "Контекст@@plural#запись(-и,-ей)": "record|records"

```

Пробелы и управляющие символы в начале и в конце локализуемых строк

Для строковых констант, которые планируется локализовать, устанавливается особое требование в использовании пробелов и [управляющих символов](#) ("`\t`" — горизонтальная табуляция; "`\n`" — перевод строки; "`\r`" — возврат каретки.) в начале и в конце таких строк:

1. Локализуемая строка не должна содержать символы пробелов в начале или в конце:

```

1 | // Неправильно
2 | rk(' Текст ')
3 | // Правильно
4 | rk('Текст')

```

2. Локализуемая строка не должна содержать символы перевода строки.

```

1 | // Неправильно
2 | rk('\nТекст')
3 | // Правильно
4 | '\n' + rk('Текст')

```

Ключи с невидимыми символами на концах строки сильно затруднят перевод. Например, строки "Сохранить",
"\nСохранить" и " Сохранить " придётся переводить отдельно.